

Antigone RH — Vue d’ensemble du projet

À lire en premier. Ce document donne la carte du territoire : ce qu’est le projet, de quoi il est composé, et comment les morceaux communiquent. Les autres fichiers zooment ensuite sur chaque partie.

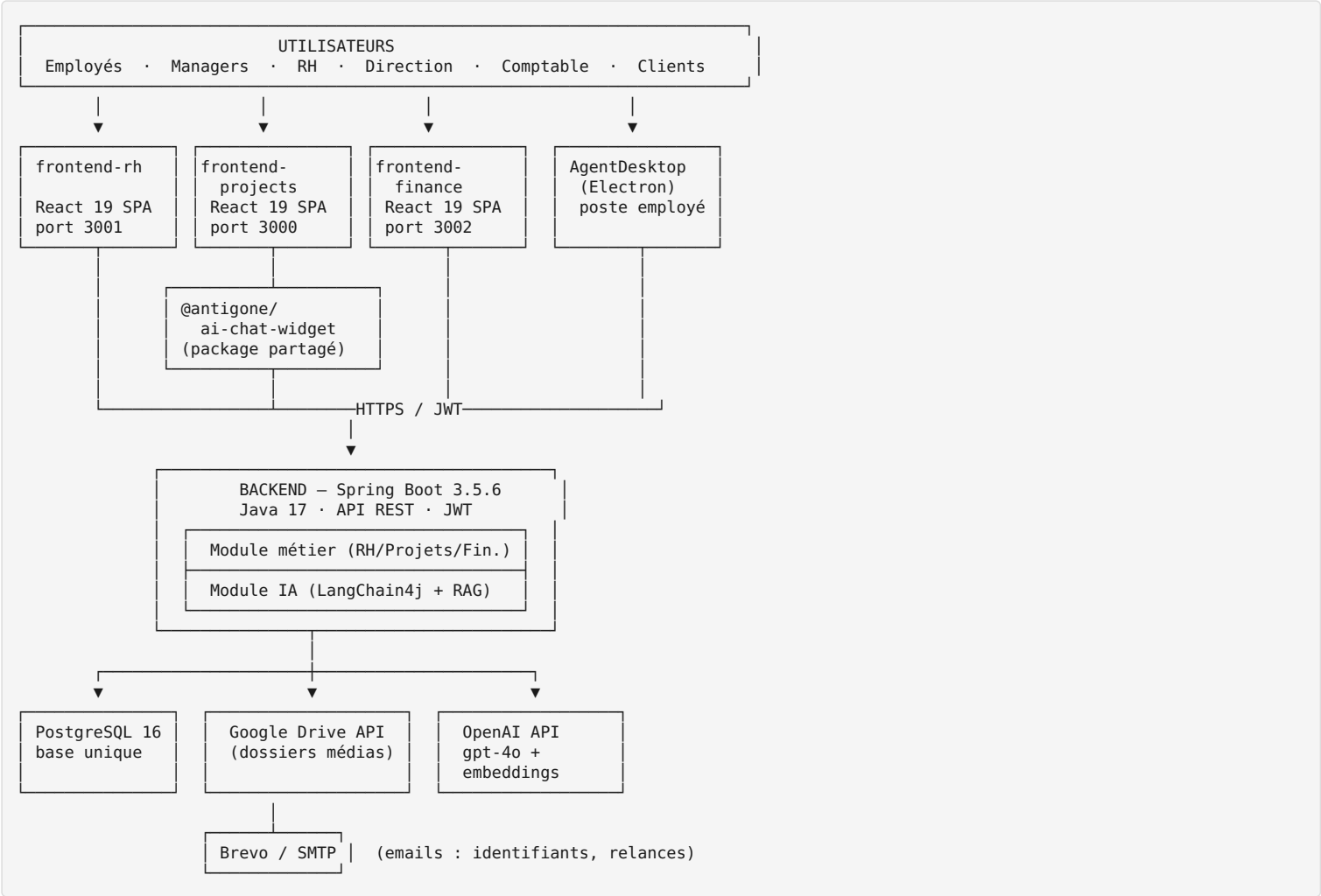
1. Le problème métier

Antigone Consulting est une agence de communication et de marketing basée à Radès (Tunisie). Comme beaucoup d’agences, elle gérait trois activités très différentes avec des outils dispersés (Excel, Drive, mails) :

Activité	Problèmes concrets
Ressources humaines	Congés sur papier, soldes calculés à la main, présence non tracée
Production / Projets	Media plans dans des fichiers Excel, pas de suivi de tâches, clients sans visibilité
Finance	Paie tunisienne calculée manuellement (CNSS + IRPP progressif), factures et relances oubliées

La solution livrée : une plateforme unique, avec **un seul backend** et **trois applications web spécialisées** (une par métier), plus **un agent de bureau** installé chez les employés, et **un assistant IA** transversal.

2. La composition du projet (les 6 briques)



Détail des 6 briques

#	Brique	Technologie	Rôle
1	Backend	Spring Boot 3.5.6, Java 17	Toute la logique métier + API REST + IA
2	frontend-projects	React 19 + Vite + TS	Projets, tâches, media plans, clients, portail client
3	frontend-rh	React 19 + Vite + TS	Employés, congés, validations, monitoring
4	frontend-finance	React 19 + Vite + TS	Factures, paie, charges, CNSS, TVA
5	AgentDesktop	Electron 28	Pointage automatique sur le poste de l'employé
6	ai-chat-widget	Package npm React partagé	Interface de chat IA réutilisée par les 3 apps

3. Pourquoi trois frontends et pas un seul ?

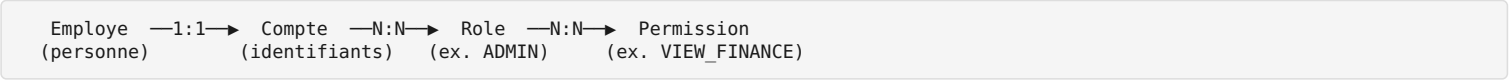
C'est la question que le jury posera. La réponse en trois points :

- Séparation métier.** Un comptable ne doit jamais voir la page « Media Plan », un social media manager ne doit jamais voir les salaires. Séparer physiquement les applications rend l'erreur impossible, alors qu'un seul gros front reposerait uniquement sur des `if`.
- Poids et temps de chargement.** L'application Finance ne charge pas Three.js (le logo 3D), ni Ant Design, ni les composants de calendrier de projets. Chaque application n'embarque que ses dépendances (voir les `package.json` : `frontend-finance` a 7 dépendances, `frontend-projects` en a 13).
- Déploiement indépendant.** Le workflow GitHub Actions construit les trois séparément. Une régression sur Finance ne bloque pas la livraison de RH.

Le point commun : elles partagent **le même backend, le même JWT, le même compte utilisateur**. On passe de l'une à l'autre sans se reconnecter (voir §5).

4. Le modèle de sécurité : rôles + permissions

C'est le cœur de l'architecture applicative. Trois notions à ne pas confondre :



- Employé** = la personne physique (nom, matricule, salaire, poste, manager...).
- Compte** = ses identifiants de connexion (`username` , `passwordHash` en BCrypt).
- Rôle** = un groupe nommé, **créé dynamiquement en base** par l'administrateur.
- Permission** = un droit atomique, ex. `VIEW_EMPLOYES` , `VIEW_FINANCE` , `VIEW_MEDIA_PLAN` .

Les 26+ permissions du catalogue (initialisées par `RoleService.initPermissions()`) :

VIEW_DASHBOARD	VIEW_EMPLOYES	VIEW_DEMANDES
VIEW_MES_DEMANDES	VIEW_VALIDATIONS	VIEW_CALENDRIER
VIEW_MON_CALENDRIER	VIEW_RESTRICTION_CONGE	VIEW_REFERENTIELS
VIEW_COMPTE	VIEW_ROLES	VIEW_DASHBOARD_RH
VIEW_MONITORING	VIEW_PROJETS	VIEW_MES_PROJETS
VIEW_TOUS_PROJETS	VIEW_EQUIPES	VIEW_CLIENTS
VIEW_MEDIA_PLAN	VIEW_TOUS_MEDIA_PLAN	VIEW_CALENDRIER_PROJETS
VIEW_DEADLINES	VIEW_REUNIONS	VIEW_FINANCE
VIEW_ASSISTANT_RH	VIEW_PROJETS_CREATE_TACHES	
MANAGE_CLIENTS	MANAGE_ALL_PROJETS	

Comment ça circule :

- L'utilisateur se connecte → le backend produit un **JWT** qui contient `roles` **et** `permissions` (voir `JwtService.generateToken`).
- Le frontend décode ce JWT et **masque les menus** auxquels l'utilisateur n'a pas droit (`PermissionRoute` dans `App.tsx` , filtrage

- du `Sidebar`).
3. Le backend **revérifie** systématiquement côté serveur (`SecurityConfig` + annotations `@PreAuthorize`). Le masquage frontend est du confort, pas de la sécurité.

Phrase à retenir pour la soutenance : « Le frontend cache, le backend interdit. Les deux sont nécessaires : l'un pour l'ergonomie, l'autre pour la sécurité. »

5. Le SSO maison entre les trois applications

Problème : les 3 applications sont sur **3 origines différentes** (`/rh/` , `/projects/` , `/finance/` en production). Le `localStorage` n'est pas partagé entre origines différentes.

La solution implémentée (`utils/authStorage.ts` , fonction `relayAuthSnapshotForSwitch`) :

1. L'utilisateur clique sur `AppSwitchButton` (par ex. RH → Finance).
2. Avant la redirection, l'application encode `{user, loginTime}` en **Base64** et l'écrit dans `window.name` .
3. `window.name` est une des rares propriétés du navigateur qui **survit à une navigation inter-origines** dans le même onglet.
4. L'application cible lit `window.name` au démarrage, le decode, l'enregistre dans son propre `localStorage` , **puis efface** `window.name` (pour ne pas laisser traîner le jeton).

Contrôles supplémentaires : - Session locale expirée après **4 h** (`SESSION_DURATION_MS`). - Expiration du JWT vérifiée aussi (`tokenExpiresAt`), côté client, avant tout usage. - L'intercepteur Axios purge la session sur toute réponse **401**.

6. Le flux de bout en bout : un exemple concret

Scénario : Sarah, social media manager, demande un congé.

1. Sarah ouvre frontend-rh → /mes-demandes → clic « Nouvelle demande »

2. Le front appelle GET /api/demandes/solde-conge/{id}
→ le backend calcule son solde (ancienneté : 18 j la 1re année, 24 j ensuite)

3. Elle saisit du 10 au 14 mars. Le front appelle
POST /api/demandes/compute-effective-days
→ le backend retire les week-ends, les jours fériés du Calendrier entreprise,
et applique la règle « congé maladie » si besoin → 5 jours ouvrés

4. POST /api/demandes → DemandeService.create()
→ crée une entité Conge (héritage JOINED depuis Demande)
→ statut EN ATTENTE
→ NotificationService prévient le manager
→ HistoriqueStatut enregistre la création

5. Le manager ouvre /validations, approuve
→ DemandeService.approve() : statut APPROUVEE, solde décrémenté,
ligne écrite dans le Calendrier, notification envoyée à Sarah

6. Le jour J, AgentDesktop détecte que Sarah est en congé (via /api/agent/config)
→ il ne compte pas d'absence injustifiée

7. En fin de mois, PayrollService voit le congé approuvé
→ pas de déduction d'absence sur le bulletin de paie

Ce scénario traverse **4 des 6 briques** — c'est un excellent fil rouge de démonstration.

7. Chiffres clés du projet (à citer en soutenance)

Élément	Nombre
Entités JPA (tables)	52 métier + 5 IA = 57
Repositories Spring Data	52 + 5
Contrôleurs REST	35 métier + 5 IA = 40
Services métier	44 + ~10 IA
DTOs	98 + 16 IA

Énumérations métier	20
Permissions	26+
Pages React	46 (projects) + 41 (RH) + 9 (finance) = 96
Outils IA exposés au LLM	12
Tâches planifiées (cron)	7

8. Déploiement

Composant	Hébergement	Détail
Backend	Render (Docker, région Frankfurt)	<code>render.yaml</code> , profil <code>prod</code>
Base de données	Render PostgreSQL 16	plan gratuit
Les 3 frontends	GitHub Pages	via GitHub Actions, sous-dossiers <code>/rh/</code> , <code>/projects/</code> , <code>/finance/</code>
AgentDesktop	Installeur <code>.exe</code> (NSIS) / <code>.dmg</code>	<code>electron-builder</code>

Astuce technique du déploiement front : GitHub Pages ne connaît pas le routage côté client. Le build copie `index.html` en `404.html` (`npm run build`), et un script dans `404.html` réécrit l'URL pour que React Router retrouve sa route.

9. Guide de lecture des autres documents

Fichier	Contenu
<code>01-BACKEND.md</code>	Le backend en détail complet : couches, sécurité, chaque module métier, la paie tunisienne, les schedulers
<code>02-FRONTEND-PROJECTS.md</code>	L'app Projets : media plans, tâches, portail client
<code>03-FRONTEND-RH.md</code>	L'app RH : congés, employés, monitoring
<code>04-FRONTEND-FINANCE.md</code>	L'app Finance : factures, paie, TVA, CNSS
<code>05-IA-EXPLIQUEE-SIMPLEMENT.md</code>	Toute l'IA expliquée avec des mots simples
<code>06-QUESTIONS-JURY.md</code>	Les questions probables + réponses préparées